NIFTY-50 Investment Intelligence Platform

Technical Report

This document describes the full analytical pipeline of the submitted platform: raw data ingestion and cleaning, technical indicator feature engineering, supervised machine learning for directional prediction, quantitative risk and portfolio analytics, and market anomaly detection. Every claim in this report corresponds directly to code in the submitted codebase. No live data, external APIs, or proprietary data sources are used.

`Dataset`	NIFTY-50 Stock Market Data (Kaggle) — Jan 2000 to Apr 2021
`Scope`	50 large-cap NSE-listed companies across multiple sectors
`Stack`	Python, Pandas, Scikit-learn, Streamlit, Plotly
`Modules`	data_pipeline.py, predictor.py, risk_analytics.py, app.py, combine_data.py
`Constraint`	Only the provided Kaggle dataset is used throughout

SECTION 1

Introduction

The platform transforms raw historical NIFTY-50 price and volume data into a multi-component investment decision-support system. The architecture has five layers: (1) data ingestion and cleaning, (2) technical indicator feature engineering, (3) supervised ML directional prediction, (4) quantitative risk and portfolio analytics, and (5) an interactive Streamlit front-end with anomaly detection. All computations are strictly historical. No live market data, external APIs, or proprietary financial data sources are used.

SECTION 2

Exploratory Data Analysis

2.1 Dataset Overview

The dataset is the NIFTY-50 Stock Market Dataset from Kaggle (rohanrao/nifty50-stock-market-data). It contains daily OHLCV records for companies listed on the NSE from January 2000 to April 2021. Individual stock files typically span approximately 4,000–4,200 trading days each. The dataset is distributed as one CSV per ticker symbol; combine_data.py merges these into NIFTY50_all.csv by inferring the Symbol from each filename.

Column	Type	Used in pipeline	Description
`Date`	datetime	Yes	Trading date (parsed via pd.to_datetime)
`Symbol`	string	Yes	NSE ticker (added by combine_data.py)
`Series`	string	No	Equity series — always "EQ" in this dataset
`Prev Close`	float	No	Previous session closing price
`Open`	float	No	Opening price (INR)
`High`	float	No	Intraday high (INR)
`Low`	float	No	Intraday low (INR)
`Last`	float	No	Last traded price (INR)
`Close`	float	Yes — primary	Closing price (INR) — all indicators derived here
`VWAP`	float	No	Volume-weighted average price
`Volume`	int	Yes — filter + features	Shares traded; used in volume ratio feature
`Turnover`	float	No	Traded value (INR)
`Trades`	float	No — 41% missing	Number of trades; substantial missing data
`Deliverable Volume`	int	No	Shares resulting in delivery
`%Deliverable`	float	No	Delivered volume as % of total volume

Table 1. Full dataset column schema with pipeline usage.

2.2 Data Cleaning (load_raw_data)

Step	Operation	Rationale

1	`to_datetime(errors='coerce'); dropna(Date, Symbol, Close)`	Removes unparseable timestamps and rows with missing core fields
2	`sort_values(Symbol, Date); reset_index`	Guarantees chronological order per ticker before any rolling calculation
3	`Drop rows: 1-day return < −70% or > +200%`	Filters unadjusted stock splits and data logging errors
4	`Drop rows: Volume ≤ 100`	Removes non-trading sessions and near-zero-liquidity days

Table 2. Data cleaning steps and rationale.

SECTION 3

Feature Engineering

All features are computed using pandas groupby('Symbol').transform(), which applies each calculation within a ticker's own time series independently. This prevents cross-symbol data leakage regardless of row ordering in the combined dataframe. The pipeline calls seven functions in sequence inside run_pipeline().

3.1 Moving Averages

Feature	Computation
SMA_5	rolling(window=5).mean() on Close
SMA_20	rolling(window=20).mean() on Close
SMA_50	rolling(window=50).mean() on Close — sets the minimum lookback (50 rows per symbol)
EMA_5	ewm(span=5, adjust=False).mean() on Close
EMA_20	ewm(span=20, adjust=False).mean() on Close

Table 3. Moving average features.

3.2 Bollinger Bands

Feature	Formula
BB_Middle	20-day rolling mean of Close
BB_High	BB_Middle + 2 × 20-day rolling std(Close)
BB_Low	BB_Middle – 2 × 20-day rolling std(Close)
BB_Width	(BB_High – BB_Low) / BB_Middle [model input only]

Table 4. Bollinger Band features. BB_Width is the only Bollinger input to the model; BB_High and BB_Low are retained for the price chart only.

3.3 RSI — Wilder's EWM Method (period = 14)

Wilder's smoothing uses ewm(com = period – 1), giving a decay factor α = 1/period = 1/14. This matches the original RSI specification. A simple rolling(14).mean() produces a different, non-standard indicator.

Step	Code
1	`delta = Close.diff()`
2	`gain = delta.clip(lower=0); loss = –delta.clip(upper=0)`
3	`avg_gain = gain.ewm(com=13, min_periods=14).mean()`
4	`avg_loss = loss.ewm(com=13, min_periods=14).mean()`
5	`RS = avg_gain / (avg_loss + 1e-9)`
6	`RSI = 100 – 100 / (1 + RS)`

Table 5. RSI computation steps (Wilder's smoothing).

3.4 MACD

Feature	Formula

`MACD_Line`	EMA(Close, span=12) − EMA(Close, span=26)
`MACD_Signal`	EMA(MACD_Line, span=9)
`MACD_Hist`	MACD_Line − MACD_Signal

Table 6. MACD features.

3.5 Volatility, Momentum & Volume

Feature	Formula
`Daily_Return`	Close.pct_change()
`Volatility_20`	rolling(20).std(Daily_Return) × √252 [annualised]
`ROC_10`	Close.pct_change(periods=10)
`Volume_SMA_20`	rolling(20).mean() on Volume
`Volume_Ratio`	Volume / Volume_SMA_20

Table 7. Volatility, momentum, and volume features.

3.6 ML Target Generation

Two targets are created by shifting Daily_Return by −1 within each symbol group. NaN values on the final row per symbol are preserved using np.where() so the pipeline's final dropna removes them cleanly. This prevents a silent NaN-to-zero conversion that would inject false DOWN labels into training data.

Target	Type	Definition
`Target_Dir`	Binary (0/1)	1 if next-day return > 0, else 0; NaN on last row per symbol
`Target_Return`	Float	Exact next-day Daily_Return (regression reference, not modelled)

Table 8. ML target columns. Final dropna requires non-null values in: SMA_50, BB_High, RSI, MACD_Line, Volatility_20, Target_Return, Target_Dir. This removes the rolling lookback rows (up to 50 per symbol) and the final shifted row.

SECTION 4

Methodology

The task is framed as binary classification: predict whether a stock's closing price will be higher the next trading day (Target_Dir = 1) or not (Target_Dir = 0). Models are trained per ticker in isolation; no cross-asset features are used as model inputs. All splits respect chronological order — no shuffling is applied at any stage. The 80% training window is subdivided into five expanding folds using TimeSeriesSplit for cross-validation, producing a mean accuracy and standard deviation that is more credible than a single holdout split.

This design choice prevents the two most common sources of inflated accuracy in financial ML: (a) random shuffling, which lets the model see future prices during training, and (b) cross-symbol feature contamination, which mixes return histories across unrelated companies. Neither is present in this implementation.

SECTION 5

Model Architecture

5.1 Algorithm

RandomForestClassifier (sklearn.ensemble). An ensemble of 100 decision trees provides implicit feature interaction modelling, built-in Gini importance scores for explainability, and robustness to non-stationary financial series. max_depth=5 constrains each tree to reduce overfitting on noisy price data.

Parameter	Value	Rationale
`n_estimators`	`100`	Standard ensemble size
`max_depth`	`5`	Shallow trees reduce overfitting on financial noise
`class_weight`	`'balanced'`	Adjusts for class imbalance in up/down days
`random_state`	`42`	Reproducibility across runs

Table 9. Model hyperparameters.

5.2 Input Features (12 total)

Feature(s)	Category
`SMA_5, SMA_20, SMA_50`	Trend — Simple Moving Averages
`EMA_5, EMA_20`	Trend — Exponential Moving Averages
`BB_Width`	Volatility — Bollinger Band Width
`RSI`	Momentum — 14-day Relative Strength Index
`MACD_Line, MACD_Hist`	Momentum — MACD line and histogram
`Volatility_20`	Risk — 20-day annualised rolling volatility
`ROC_10`	Momentum — 10-day Rate of Change
`Volume_Ratio`	Volume — relative to 20-day average

Table 10. Model input features.

5.3 Training & Validation Protocol

Stage	Detail
`NaN guard`	dropna(features + ['Target_Dir']); reset_index before splitting
`Chronological split`	80% training window / 20% holdout test — no shuffling
`Cross-validation`	5-fold TimeSeriesSplit on training window; each fold expands forward in time
`Primary metric`	Mean CV accuracy across 5 folds (std also computed and logged)
`Secondary metrics`	Holdout accuracy; precision_score(zero_division=0) on UP calls
`Model persistence`	Optional joblib.dump to {symbol}_rf_model.pkl when save_dir is provided

Table 11. Training and validation protocol.

SECTION 6

Portfolio Construction Logic

Three investor profiles are supported. Each selects exactly three stocks from the computed metrics dataframe using a different quantitative criterion. Allocation weights are then computed via inverse-volatility weighting rather than fixed percentages.

Profile	Selection method	Criterion
Conservative	`nsmallest(3, 'Annualized_Volatility')`	Lowest historical price volatility
Balanced	`nlargest(3, 'Sharpe_Ratio')`	Highest risk-adjusted return (Sharpe)
Aggressive	`nlargest(3, 'Cum_Return')`	Highest total historical cumulative return

Table 12. Portfolio profile selection criteria.

6.1 Inverse-Volatility Weighting

After selecting the three stocks, weights are computed as: w■ = (1/vol■) / Σ■(1/vol■). Lower-volatility stocks receive proportionally higher allocations. Normalising by the sum guarantees weights sum exactly to 1 and handles edge cases with fewer than three qualifying stocks without additional guard logic.

6.2 Portfolio Visualisation

Three outputs are rendered per profile: (a) a formatted allocation table with percentage weights, (b) a Plotly donut chart of capital weights, and (c) a full-width Pearson correlation heatmap of the selected stocks' daily returns to show diversification benefit.

SECTION 7

Risk Assessment Methodology

compute_asset_metrics() computes five metrics for every symbol with at least 50 records. The annual risk-free rate is 6% (broadly aligned with the RBI repo rate range over the dataset period), converted to a daily equivalent via compound interest: $r_{f,daily} = (1.06)^{1/252} - 1$.

Metric	Formula	Interpretation
Cumulative Return	`prod(1 + r) − 1`	Total compounded return over full available history
Annualised Volatility	`std(r) × √252`	Annualised standard deviation of daily returns
Sharpe Ratio	`(mean(e) / std(e)) × √252` where `e = r − r_f`	Return per unit of total risk; penalises upside and downside volatility equally
Sortino Ratio	`(mean(e) / std(e[e<0])) × √252`	Return per unit of downside risk only; upside volatility is not penalised
Maximum Drawdown	`min((V − peak) / peak)`	Worst peak-to-trough loss over the full price history

Table 13. Risk metrics, formulas, and interpretation.

Sharpe penalises all volatility, including upside moves. For the NIFTY-50 universe, which includes high-growth names with positive return skew, Sortino provides a more informative picture. Both are computed and displayed side by side in the application.

SECTION 8

Explainability Techniques

The platform uses the Random Forest's built-in Gini importance scores (model.feature_importances_). Each value represents the mean decrease in Gini impurity attributed to that feature across all 100 trees. Features with higher scores had more discriminative power separating UP from DOWN days during training.

Attribute	Detail
`Importance type`	Gini impurity reduction, averaged across all 100 trees
`Output`	DataFrame sorted descending by Importance: Feature, Importance
`Display`	Top 6 features as horizontal Plotly bar chart with Viridis colour scale
`Scope`	Per-stock; re-computed fresh on each training run
`Limitation`	Gini importance can favour high-cardinality continuous features; it is a correlation-based measure, not a causal attribution

Table 14. Explainability implementation details.

SECTION 9

Market Anomaly Detection

Trading days are flagged as anomalies when the absolute deviation of Daily_Return from the stock's own historical mean exceeds three standard deviations: |r – mean(r)| > 3 × std(r). This threshold is a standard statistical criterion for detecting extreme events such as market crashes, short squeezes, earnings surprises, and data recording errors. The per-stock mean and std are computed on non-NaN returns only.

Step	Operation
1	Compute mean and std of Daily_Return for the selected stock (NaN rows excluded)
2	Flag rows where | Daily_Return – mean | > 3 × std
3	Overlay flagged days as red circle markers on a Plotly price-history line chart
4	Display sortable table of anomaly dates, Close price, and Daily_Return

Table 15. Anomaly detection procedure.

SECTION 10

Key Insights

10.1 Notable Design Decisions

Decision	Detail
No look-ahead bias	groupby.transform() prevents cross-symbol contamination. Chronological splits and TimeSeriesSplit CV prevent temporal leakage. These are the two most common sources of inflated accuracy in financial ML.
Wilder's RSI	ewm(com=13) matches the published RSI specification. The difference from rolling(14).mean() is visible at short windows and affects overbought/oversold signal timing.

Inverse-vol weights	Replacing fixed percentage allocations with 1/vol normalised weights makes capital allocation respond to each stock's actual measured risk.
Sortino alongside Sharpe	For equity markets with positive return skew, Sortino is more informative than Sharpe. Computing both gives a more complete risk picture than either alone.
NaN label safety	np.where() in generate_ml_targets() preserves NaN on the last row per symbol so it is removed by dropna rather than silently labelled as DOWN (= 0).

Table 16. Notable design decisions.

10.2 Limitations

• Target_Return exists in the dataframe but no regression model is trained on it. Only directional classification (Target_Dir) is modelled.

• Cross-validation accuracy on large-cap NIFTY-50 stocks typically falls close to 50–52%, which is consistent with the semi-strong form of the Efficient Market Hypothesis. The platform's value lies in the decision-support framework and risk analytics, not in claiming exploitable predictive edge.

• The dataset ends April 2021. Results do not reflect market conditions since then.

• Models are trained per ticker in isolation; no cross-asset correlation features are used as inputs.

• Portfolio selection draws from three stocks per profile with no sector-diversity constraint. Selected stocks could theoretically all belong to the same sector.

• Portfolio returns are computed on historical data only. Transaction costs, brokerage fees, and slippage are not modelled.

• The 3-sigma anomaly threshold is a statistical heuristic and does not distinguish genuine market events from data recording errors.

• Open, High, Low, VWAP, Prev Close, and Deliverable Volume columns are available in the dataset but are not currently used as model inputs.